

API Routes — Route Handlers

Pages Router vs App Router · CRUD · CORS · Caching · Streaming · File Upload

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is an API Route?

An API Route is a **backend HTTP endpoint built inside your Next.js project**. No separate Express or Node server needed. Create a route.ts file, export named functions for each HTTP method, and Next.js automatically exposes them as public API endpoints.

Without API Routes:

Next.js frontend → calls → Separate Express server

Two projects, two deployments, two repos

With API Routes:

app/api/products/route.ts → accessible at /api/products

Frontend + Backend = ONE project ■

Pages Router vs App Router Syntax

PAGES ROUTER (Legacy)

```
// pages/api/products.ts
import { NextApiRequest,
  NextApiResponse } from "next";

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method === "GET") {
    res.status(200).json(data);
  }
  else if (req.method === "POST") {
    const body = req.body;
    res.status(201).json({...});
  }
  else {
    res.status(405)
      .end("Not Allowed");
  }
}
```

File Structure → URL Mapping

```
else {
  res.status(405)
    .end("Not Allowed");
}
```

APP ROUTER (Current Standard)

```
// app/api/products/route.ts
import { NextRequest,
  NextResponse } from "next/server";

export async function GET() {
  return NextResponse.json(data);
}

export async function POST(
  request: NextRequest
) {
  const body = await request.json();
  return NextResponse.json(
    body, { status: 201 }
  );
}
// DELETE not exported?
// Auto returns 405 ■
```

```

app/
├── api/
│   ├── products/
│   │   ├── route.ts           → /api/products
│   │   └── [id]/
│   │       ├── route.ts       → /api/products/123
│   │       └── crops/
│   │           ├── route.ts    → /api/crops
│   │           └── [id]/
│   │               ├── route.ts → /api/crops/456
│   │               └── prices/
│   │                   └── route.ts → /api/crops/456/prices
│   └── upload/
│       └── route.ts           → /api/upload

```

All Supported HTTP Methods

```

// app/api/products/route.ts
import { NextRequest, NextResponse } from 'next/server';

export async function GET(request: NextRequest) {} // Read data
export async function POST(request: NextRequest) {} // Create
export async function PUT(request: NextRequest) {} // Replace all
export async function PATCH(request: NextRequest) {} // Partial update
export async function DELETE(request: NextRequest) {} // Delete
export async function HEAD(request: NextRequest) {} // Like GET, no body
export async function OPTIONS(request: NextRequest) {} // CORS preflight

// If a method is NOT exported:
// Next.js automatically returns 405 Method Not Allowed

```

Basic Route Handler — GET and POST

```

// app/api/products/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@lib/db';

// GET /api/products
export async function GET() {
  const products = await db.product.findMany();
  return NextResponse.json(products);
}

// POST /api/products
export async function POST(request: NextRequest) {
  const body = await request.json();
  const product = await db.product.create({ data: body });
  return NextResponse.json(product, { status: 201 });
}

```

■ INTERMEDIATE LEVEL

Dynamic Route Handlers — [id]

```
// app/api/products/[id]/route.ts

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } } // id from URL
) {
  const product = await db.product.findUnique({
    where: { id: params.id }
  });
  if (!product) {
    return NextResponse.json({ error: 'Not found' }, { status: 404 });
  }
  return NextResponse.json(product);
}

export async function PUT(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  const updated = await db.product.update({
    where: { id: params.id }, data: body
  });
  return NextResponse.json(updated);
}

export async function DELETE(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  await db.product.delete({ where: { id: params.id } });
  return NextResponse.json({ message: 'Deleted' });
}
```

Reading Request Data — All Ways

```

export async function POST(request: NextRequest) {

  // 1. JSON body
  const body = await request.json();
  // body = { name: 'wheat', price: 22 }

  // 2. Query parameters (?page=2&category=seeds)
  const { searchParams } = new URL(request.url);
  const page      = searchParams.get('page')      ?? '1';
  const category = searchParams.get('category') ?? '';

  // 3. Headers
  const authHeader = request.headers.get('authorization');
  const contentType = request.headers.get('content-type');

  // 4. FormData (file uploads)
  const formData = await request.formData();
  const file = formData.get('file') as File;
  const name = formData.get('name') as string;

  // 5. Cookies
  const token = request.cookies.get('auth-token')?.value;
}

```

Sending Responses — All Ways

```

// 1. JSON response (default 200)
return NextResponse.json({ message: 'Success' });

// 2. JSON with custom status code
return NextResponse.json({ error: 'Not found' }, { status: 404 });

// 3. JSON with custom headers
return NextResponse.json(data, {
  status: 200,
  headers: { 'Cache-Control': 'public, max-age=3600' }
});

// 4. Redirect
return NextResponse.redirect(new URL('/login', request.url));

// 5. Plain text
return new Response('Hello World', { status: 200 });

// 6. Empty response (204 No Content)
return new Response(null, { status: 204 });

```

CORS — Cross Origin Resource Sharing

When your API is called from a different domain (mobile app, external client), you need CORS headers. The browser sends an **OPTIONS preflight** request first — you must handle it.

```
// app/api/products/route.ts

// Reusable CORS headers
const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type, Authorization',
};

// Handle preflight (browser sends OPTIONS before actual request)
export async function OPTIONS() {
  return NextResponse.json({}, { headers: corsHeaders });
}

export async function GET() {
  const products = await db.product.findMany();
  return NextResponse.json(products, { headers: corsHeaders });
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  const product = await db.product.create({ data: body });
  return NextResponse.json(product, {
    status: 201, headers: corsHeaders
  });
}
```

Server Actions vs API Routes — Full Comparison

Feature	Server Actions	API Routes (route.ts)
Best for	Mutations from your own UI	Public endpoints, mobile, webhooks
Called by	Your Next.js app only	Anyone with the URL
HTTP Method	Always POST	Any method (GET/POST/PUT/DELETE)
CSRF Protection	Automatic	Must implement manually
Works without JS	Yes (native form)	No
Type safety	End-to-end automatic	Manual / Zod needed
Streaming response	No	Yes
File upload	Yes (FormData)	Yes
Boilerplate	Very low	Medium

■ EXPERT LEVEL

Route Handler Caching — GET Requests

GET Route Handlers in App Router are **statically cached by default** when they do not use the request object or dynamic functions. This is a common gotcha — you may get stale data if you forget to opt out.

```
// CACHED by default (static) – may serve stale data!
export async function GET() {
  const data = await db.product.findMany();
  return NextResponse.json(data);
}

// DYNAMIC – using request object opts out of cache
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url); // dynamic!
  const data = await db.product.findMany();
  return NextResponse.json(data);
}

// FORCE DYNAMIC – always fresh (like no-store)
export const dynamic = 'force-dynamic';
export async function GET() {
  const data = await db.product.findMany();
  return NextResponse.json(data);
}

// ISR – cache with hourly revalidation
export const revalidate = 3600;
export async function GET() {
  const data = await db.product.findMany();
  return NextResponse.json(data);
}

// POST, PUT, DELETE are NEVER cached (always dynamic)
```

Reusable Auth Middleware Pattern

```

// lib/api-middleware.ts
import { NextRequest, NextResponse } from 'next/server';
import { auth } from '@clerk/nextjs/server';

// Reusable wrapper – adds auth to any route handler
export function withAuth(
  handler: (req: NextRequest, ctx: any, userId: string)
    => Promise<NextResponse>
) {
  return async (req: NextRequest, ctx: any) => {
    const { userId } = await auth();
    if (!userId) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }
    return handler(req, ctx, userId);
  };
}

// app/api/crops/route.ts – clean, auth handled by wrapper
import { withAuth } from '@lib/api-middleware';

export const POST = withAuth(async (request, ctx, userId) => {
  const body = await request.json();
  const crop = await db.crop.create({
    data: { ...body, createdBy: userId }
  });
  return NextResponse.json(crop, { status: 201 });
});

```

Centralized Error Handling

```

// lib/api-error.ts
import { NextResponse } from 'next/server';

export class ApiError extends Error {
  constructor(public statusCode: number, message: string) {
    super(message);
  }
}

export function handleApiError(error: unknown) {
  console.error("API Error:", error);

  if (error instanceof ApiError) {
    return NextResponse.json(
      { error: error.message }, { status: error.statusCode }
    );
  }
  // Prisma not found error
  if ((error as any)?.code === "P2025") {
    return NextResponse.json({ error: 'Not found' }, { status: 404 });
  }
  // Generic fallback
  return NextResponse.json({ error: 'Server error' }, { status: 500 });
}

// Usage in any route:
export async function GET(request: NextRequest,
  { params }: { params: { id: string } }) {
  try {
    const crop = await db.crop.findUniqueOrThrow(
      { where: { id: params.id } }
    );
    return NextResponse.json(crop);
  } catch (error) {
    return handleApiError(error); // all errors handled centrally
  }
}

```

Streaming Response — Server-Sent Events

```
// app/api/stream/route.ts
// Useful for: AI responses, real-time updates, large data

export async function GET() {
  const encoder = new TextEncoder();

  const stream = new ReadableStream({
    async start(controller) {
      const crops = await db.crop.findMany();

      for (const crop of crops) {
        const chunk = encoder.encode(
          `data: ${JSON.stringify(crop)}\n\n`
        );
        controller.enqueue(chunk);
        await new Promise(r => setTimeout(r, 100));
      }
      controller.close();
    }
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    }
  });
}
```

■ Critical Gotchas

■ Gotcha 1: `route.ts` and `page.tsx` cannot exist in the SAME folder. They conflict. Solution: always put API routes under `app/api/` folder to completely separate them from page routes.

■ Gotcha 2: GET Route Handlers are CACHED by default in App Router. If your GET returns dynamic data, add `export const dynamic = 'force-dynamic'` or use the request object to opt out of caching. Forgetting this = stale API responses.

■ Gotcha 3: Fetch memoization does NOT apply in Route Handlers. Each `fetch()` inside a Route Handler is always a fresh network request — no deduplication. This is different from Server Components where memoization works.

■ Gotcha 4: Always return a `Response`. Not returning anything = Next.js crashes with unhandled error. Every code path must return `NextResponse.json()` or `new Response()`.

■ Best Practice: Always validate and authenticate in API Routes. Never trust incoming data. Check auth session, validate body with Zod, handle Prisma errors gracefully. Use a centralized `handleApiError()` function to avoid repetitive error handling code.

■ Best Practice: Use API Routes for public/external consumers and Server Actions for your own UI. This keeps your architecture clean — API Routes handle the public contract, Server Actions handle internal UI mutations.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is an API Route in Next.js? Why is it useful?

Answer: An API Route is a backend HTTP endpoint built inside your Next.js project — no separate Express server needed. Create a `route.ts` file inside `app/api/`, export named functions per HTTP method (GET, POST, etc.), and Next.js exposes them as real API endpoints. Useful for building full-stack apps in one project, creating public APIs, handling webhooks, and more.

BEGINNER

Q2. What is the difference between Pages Router and App Router API syntax?

Answer: Pages Router: single default export handler(`req`, `res`) inside `pages/api/` — you check `req.method` manually with `if/else`. App Router: separate named exports (GET, POST, PUT) inside `app/api/folder/route.ts` — cleaner, method-specific. App Router uses Web standard Request/Response instead of Node.js `req/res`. Unhandled methods auto-return 405. App Router is the current standard.

BEGINNER

Q3. What HTTP methods does Next.js support in Route Handlers?

Answer: GET, POST, PUT, PATCH, DELETE, HEAD, and OPTIONS. Each is a separate named export in `route.ts`. If a method is not exported, Next.js automatically returns 405 Method Not Allowed — you do not need to handle this yourself.

Intermediate Level

INTERMEDIATE

Q4. How do you read query params, request body, and headers in a Route Handler?

Answer: Query params: `new URL(request.url).searchParams.get("key")`. JSON body: `await request.json()`. FormData: `await request.formData()` for file uploads. Headers: `request.headers.get("authorization")`. Cookies: `request.cookies.get("token")?.value`.

INTERMEDIATE

Q5. When should you use API Routes vs Server Actions?

Answer: API Routes for: public endpoints consumed by mobile apps, external services, webhooks, third-party integrations — anything outside your Next.js app. Server Actions for: mutations triggered from your own UI — forms, buttons, interactions within the app. Server Actions have less boilerplate and automatic CSRF protection. API Routes support streaming, all HTTP methods, and can be called by anyone.

INTERMEDIATE**Q6. How do you handle CORS in Next.js Route Handlers?**

Answer: Export an OPTIONS handler that returns CORS headers — browsers send a preflight OPTIONS request before the actual request. Add the same CORS headers to all other method responses. Required headers: Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers. Use specific domain instead of wildcard * for production security.

INTERMEDIATE**Q7. How do you create a reusable auth check across multiple Route Handlers?**

Answer: Create a withAuth higher-order function in a lib file. It wraps any handler, checks auth() from Clerk/NextAuth, returns 401 if not authenticated, otherwise calls the inner handler passing userId. Then wrap each protected route: export const POST = withAuth(async (req, ctx, userId) => {...}). DRY principle — auth logic written once, reused everywhere.

Expert / Syngenta-Style Questions

EXPERT**Q8. Can GET Route Handlers be cached? How do you control it?**

Answer: Yes — GET Route Handlers are statically cached by default in App Router when they do not use the request object. To make dynamic (no cache): use the request parameter, or add export const dynamic = "force-dynamic". For ISR-style: export const revalidate = 3600. POST/PUT/DELETE are never cached. This is a common gotcha — forgetting to add force-dynamic on a GET handler returns stale data to all callers.

EXPERT**Q9. Can route.ts and page.tsx exist in the same folder?**

Answer: No — they conflict. A folder cannot be both a UI route (page.tsx) and an API route (route.ts). Next.js throws an error if both exist in the same directory. Convention: always put API routes under app/api/ to completely separate backend endpoints from UI pages.

SYNGENTA**Q10. Design the complete API route structure for Syngenta crop management system.**

Answer: app/api/crops/route.ts: GET (list with pagination + category filter), POST (create with auth). app/api/crops/[id]/route.ts: GET (single), PUT (update), DELETE (delete). app/api/crops/[id]/prices/route.ts: GET (price history), POST (add price point). app/api/upload/route.ts: POST (image upload for crop photos). lib/api-middleware.ts: withAuth() wrapper. lib/api-error.ts: handleApiError() including Prisma P2025 handler. All protected routes use withAuth wrapper. All errors use centralized handler.

EXPERT**Q11. What is the difference between fetch memoization in Server Components vs Route Handlers?**

Answer: In Server Components: Next.js deduplicates identical `fetch()` calls within the same render pass — layout and page calling the same URL = one network request. In Route Handlers: memoization does NOT apply because Route Handlers are outside the React component tree. Each `fetch()` inside a Route Handler is always a fresh independent request — no deduplication at all.